DC Weather & Moving Violations Data Integration Using the Google Cloud Platform

Environment: Google Cloud Platform
Database System: MySQL

1. Project Overview:

The objective of this project is to create and implement a cloud-based data integration and storage system to analyze the relationship between moving violations and daily weather in Washington, D.C. The primary objective of this project is not on analytics, but on developing a consistent, efficient, and dependable ETL (Extract-Transform-Load) pipeline that allows for continuous data updates.

The system was developed using:

- MySQL for data storage
- Python for ETL processing
- A virtual machine and managed database services on Google Cloud Platform

This implementation enables structured storage of millions of traffic records alongside daily historical weather data for long-term analysis.

2. Tools & Technologies Used:

- **Cloud Platform:** Google Cloud Platform
- **Database:** MySQL (Cloud SQL)
- **Compute:** Google Compute Engine
- **Language:** Python
- **Libraries:** Pandas, mysql-connector-python
- **Interface:** Cloud SQL Studio

3. Data Sources:

3.1. **The weather dataset** was downloaded in Excel format and the data ranges from September 2024 to November 2025 and includes around **450 daily records**. Key columns are:

- Date
- Maximum, minimum, and average temperature
- Dew point and humidity
- Precipitation amount and probability
- Wind speed and direction
- Cloud cover and visibility

3.2. **The Moving violation dataset** was downloaded as seven monthly CSV files from September 2024 to March 2025, which included more than 1.7 million records.

- Unique violation identifier (OBJECTID)
- Location and coordinates
- Issue date and time
- Issuing agency details
- Violation code and description
- Plate state
- Accident indicator
- Fine amount and penalties

4. Cloud Infrastructure Setup:

The entire project was implemented in Google Cloud Platform as follows:

- **A Google Cloud Project** was created and billing was enabled.
- **A Cloud SQL MySQL instance** was created to host the relational database.
- **Created a Database in MySQL** to store the Structured table data.
- **A Compute Engine Virtual Machine (Ubuntu Linux)** was created to run Python ETL operations.

- **Secure connectivity** was established between the VM and Cloud SQL using the database public IP.

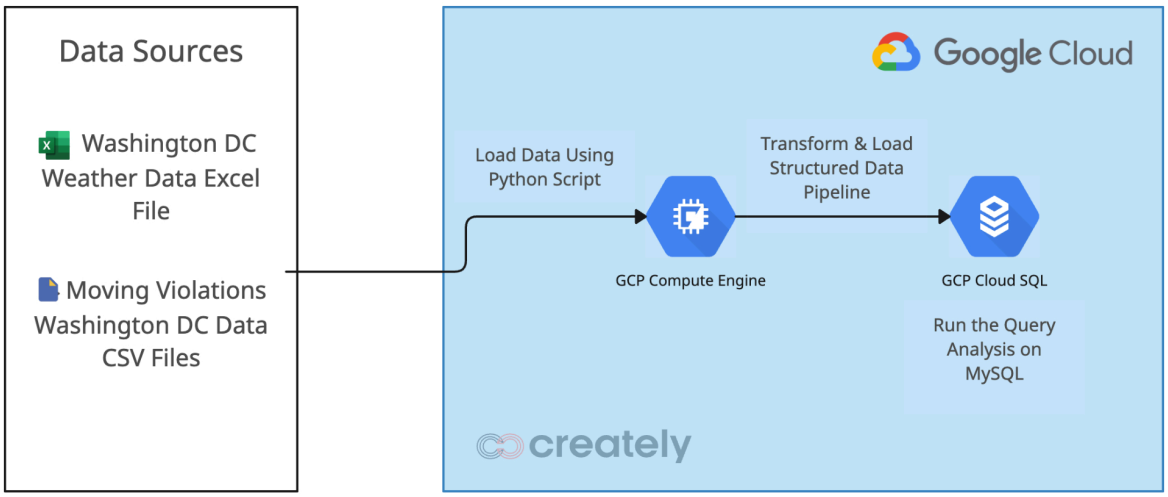
5. System Architecture:

The system follows a three-tier cloud architecture:

1. **VM Layer (Compute Engine):**
   a. Stores raw Excel/CSV files and runs Python ETL scripts.
2. **ETL Layer (Python + Pandas)**
   a. Reads, cleans, transforms, and inserts data.
3. **Database Layer (Cloud SQL – MySQL)**
   a. Stores final structured data for analysis.

Data Flow:-

CSV/Excel → Python (VM) → MySQL (Cloud SQL) → SQL Analysis



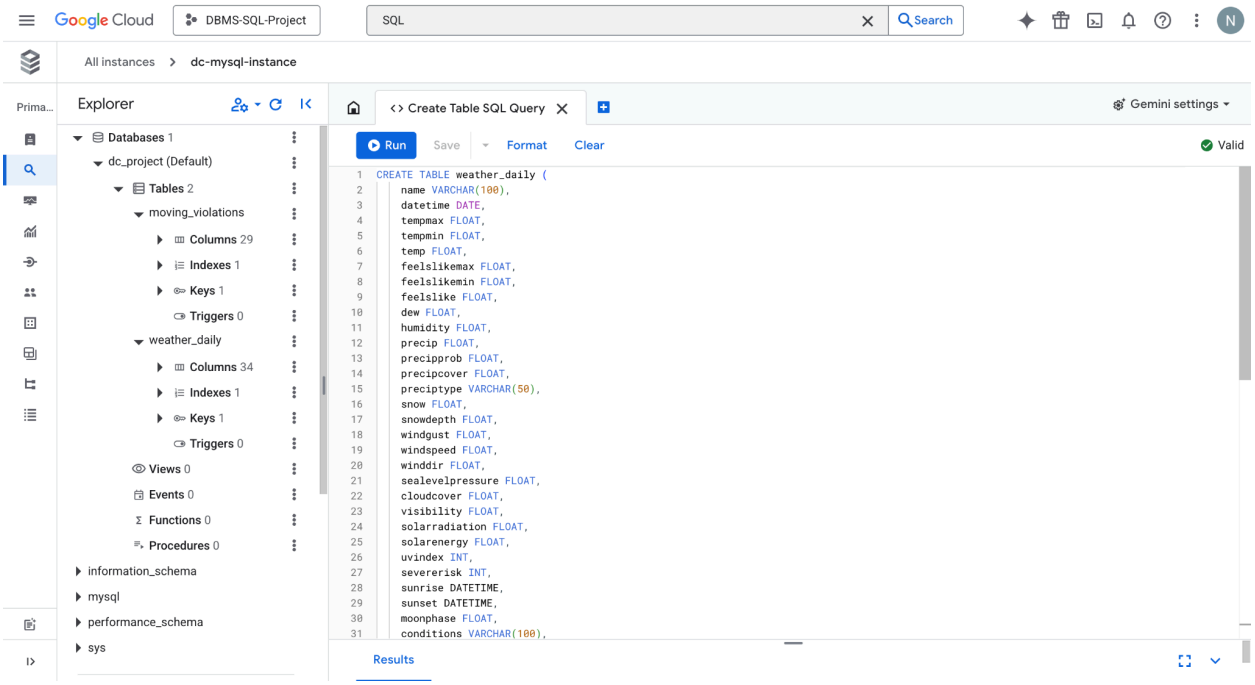
6. Database Design and Data Model:

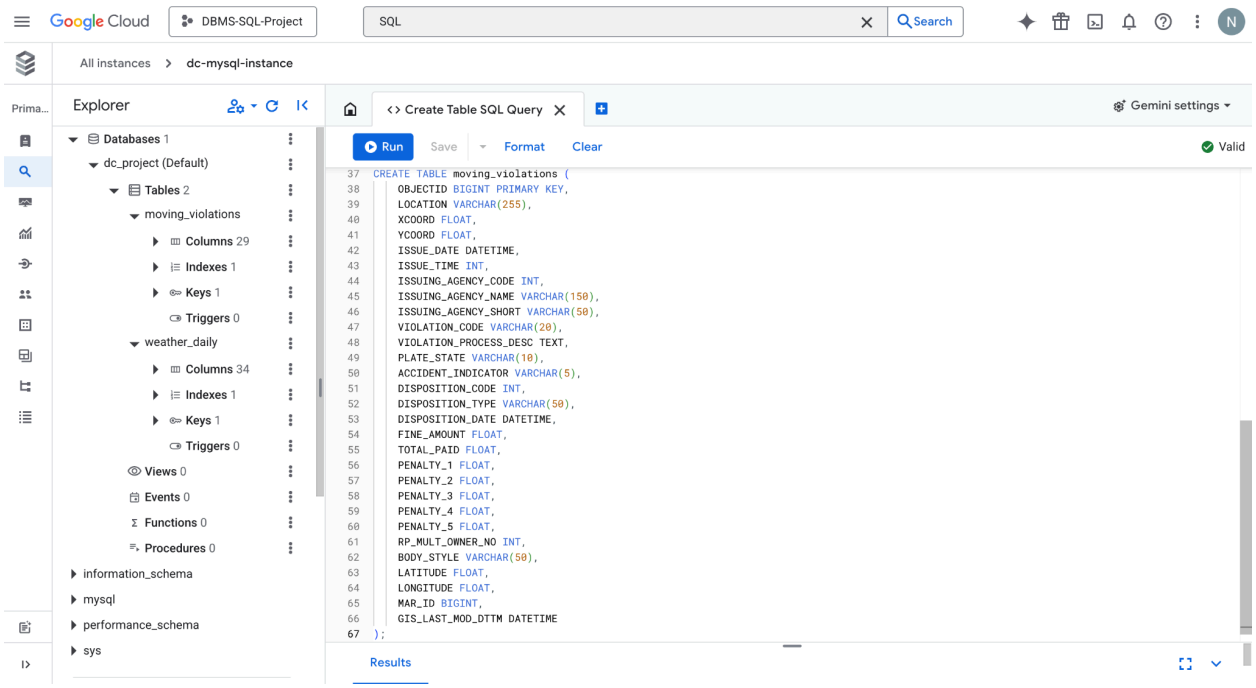
1. Created a Instance in MySQL & Created a MySQL database named:

   - **Dc_project.** The database was created to store all integrated data.

2. Tables Implemented:

- weather_daily
    - Stores one row per calendar day of weather data.
    - Includes temperature, precipitation, humidity, wind, cloud cover, and weather conditions.
- moving_violations:
    - Stores individual traffic violation records.
    - The column **OBJECTID** is defined as the **PRIMARY KEY** to ensure data integrity and **prevent duplicate records.**





3. Logical Relationship:

   The two tables are logically related by **date**:

   - Weather **datetime** <-> Violations **ISSUE_DATE**
   - This relationship supports weather-based traffic analysis while keeping both datasets normalized.

7. Compute Engine & Virtual Machine (VM) Instance Design:

To support the ETL processing and data integration tasks, a dedicated virtual machine (VM) was created using Google Compute Engine within the Google Cloud Platform environment. The VM serves as the data processing layer of the architecture, while Cloud SQL serves as the data storage layer.

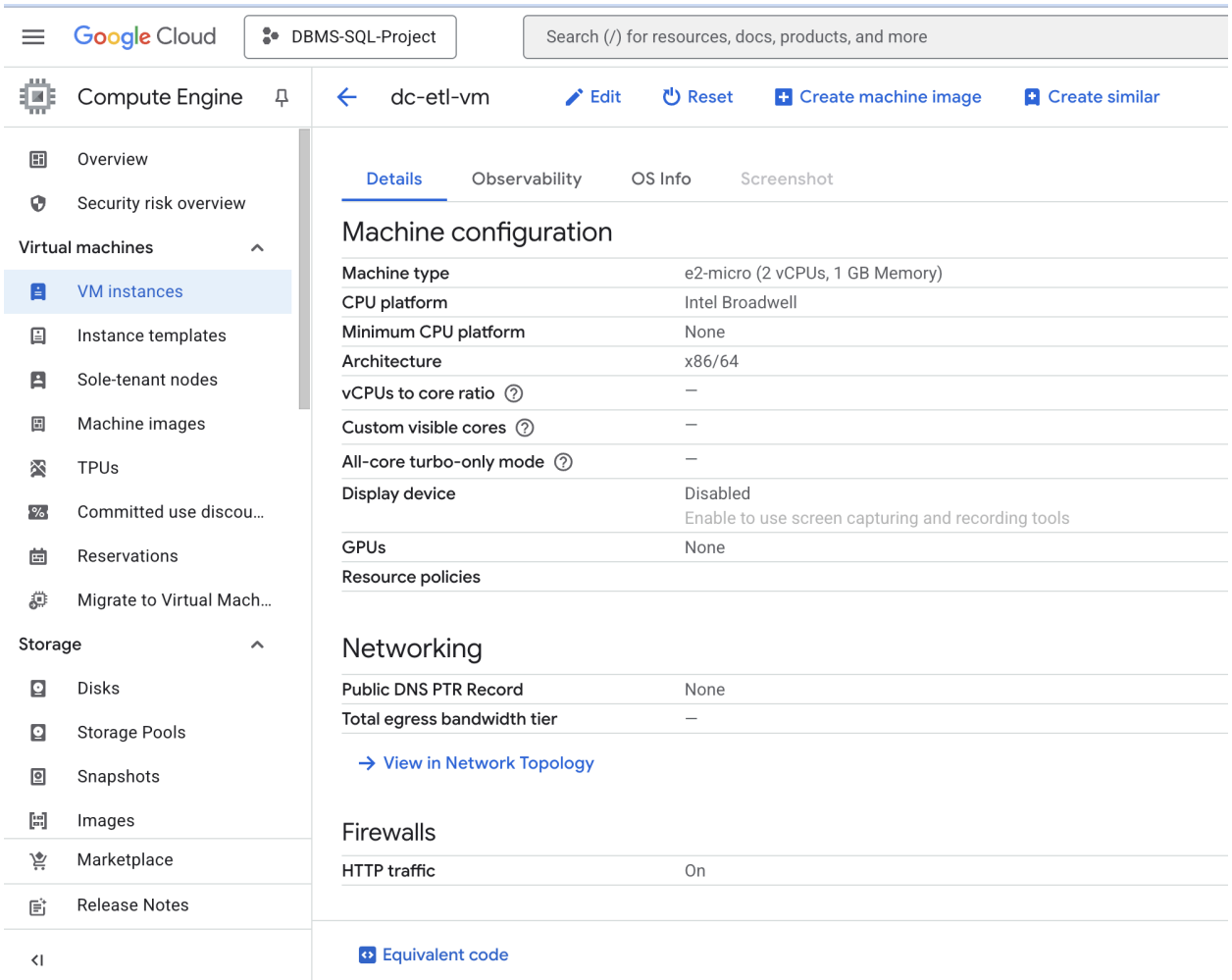
Purpose Of VM:-

The virtual machine was designed to perform the following critical functions:

- Store all raw data files (Excel and CSV datasets)
- Execute the Python-based ETL script
- Perform data cleaning and transformation

- Establish a secure remote connection to the MySQL database
- Handle large-scale data loading using chunk processing

Virtual Machine Configuration Details:-



This lightweight configuration was sufficient due to the use of chunk-based processing, which minimizes memory consumption during large file ingestion & also cost will be very less compared to the other machine types.

Software Installed:

After creating the VM Instance, I have installed the python software & python libraries as well in the VM terminal.

- Python 3.10
- Python Libraries:
    - Pandas (for data manipulation)
    - mysql-connector-python (for database connectivity)

File Management & Database Connectivity:

- Created a Python ETL script file inside the VM to perform the Database Connectivity & ETL Process.
- The VM connects to the Cloud SQL MySQL instance using:
    - Public IP address of the database
    - Database username and password
    - Python mysql.connector library

A centralized configuration object (DB_CONFIG) is used in the ETL script to manage database connectivity.

```
import pandas as pd

import mysql.connector

from datetime import datetime

DB_CONFIG = {

"host": "35.225.104.203",

"user": "dc_user",

"password": "Nagesh",

"database": "dc_project"

}

def db connection():

return mysql.connector.connect(**DB_CONFIG)
```

8. ETL (Extract–Transform–Load) Implementation:

EXTRACT:

- Weather Data:
    - The Excel file was downloaded manually from the weather provider.
    - The file was uploaded to the Google Cloud virtual machine.

- Moving Violations Data

    - Seven monthly moving_violation CSV files were downloaded separately.
    - All CSV files were uploaded to the same virtual machine for processing.

File transfer to the VM was performed using secure cloud copy commands. All files were verified on the VM before Transform & Load processing began.

```
*** System restart required ***
Last login: Wed Dec  3 02:19:53 2025 from 35.235.245.128
nageshasnn1@dc-etl-vm:~$ pwd
/home/nageshasnn1
nageshasnn1@dc-etl-vm:~$ cd dc_project
nageshasnn1@dc-etl-vm:~/dc_project$ ls
 Moving_Violations_Issued_in_December_2024.csv   Moving_Violations_Issued_in_March_2025.csv     Moving_Violations_Issued_in_September_2024.csv   etl.py.save
 Moving_Violations_Issued_in_February_2025.csv   Moving_Violations_Issued_in_November_2024.csv  'Washington, DC, Sept 2024 to Nov 2025.xlsx'     etl.py.save.1
 Moving_Violations_Issued_in_January_2025.csv    Moving_Violations_Issued_in_October_2024.csv    etl.py
nageshasnn1@dc-etl-vm:~/dc_project$
```

TRANSFORM:

- Using Python:

    - Date columns converted to proper datetime format.
    - Missing values replaced with NULL.
    - Only required columns retained.
    - Large files processed using chunk loading to prevent memory issues.

LOAD:

- Using Python:

- Cleaned weather data is inserted into the weather_daily table.
- Violation data is inserted into the moving_violations table using batch insert.
- Large violation files are processed in chunks (5000 rows at a time) to avoid VM memory crashes.

```python
# ---------------- LOAD WEATHER ----------------
def load_weather():
    weather_file = "Washington, DC, Sept 2024 to Nov 2025.xlsx"
    print("Reading Weather Data:", weather_file)

    weather_df = pd.read_excel(weather_file)
    weather_df["datetime"] = pd.to_datetime(weather_df["datetime"]).dt.date
    weather_df = weather_df.where(pd.notnull(weather_df), None)

    weather_sql = """
    INSERT INTO weather_daily (
        name, datetime, tempmax, tempmin, temp, feelslikemax, feelslikemin,
        feelslike, dew, humidity, precip, precipprob, precipcover, preciptype,
        snow, snowdepth, windgust, windspeed, winddir, sealevelpressure,
        cloudcover, visibility, solarradiation, solarenergy, uvindex,
        severerisk, sunrise, sunset, moonphase, conditions,
        description, icon, stations
    )
    VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,
            %s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,
            %s,%s,%s,%s,%s)
    """

    weather_data = weather_df[[
        "name","datetime","tempmax","tempmin","temp","feelslikemax","feelslikemin",
        "feelslike","dew","humidity","precip","precipprob","precipcover","preciptype",
        "snow","snowdepth","windgust","windspeed","winddir","sealevelpressure",
        "cloudcover","visibility","solarradiation","solarenergy","uvindex",
        "severerisk","sunrise","sunset","moonphase","conditions",
        "description","icon","stations"
    ]].values.tolist()

    db_conn = db_connection()
    db_cursor = db_conn.cursor()

    db_cursor.executemany(weather_sql, weather_data)
    db_conn.commit()

    db_cursor.close()
    db_conn.close()
    print("Weather Data Imported & Loaded successfully")
```

```python
def load_violations(file_name):
    print("Loading violations:", file_name)

    chunk_size = 5000

    db_conn = db_connection()
    db_cursor = db_conn.cursor()

    violations_sql = """
    INSERT INTO moving_violations (
        OBJECTID, LOCATION, XCOORD, YCOORD, ISSUE_DATE, ISSUE_TIME,
        ISSUING_AGENCY_CODE, ISSUING_AGENCY_NAME, ISSUING_AGENCY_SHORT,
        VIOLATION_CODE, VIOLATION_PROCESS_DESC, PLATE_STATE,
        ACCIDENT_INDICATOR, DISPOSITION_CODE, DISPOSITION_TYPE,
        DISPOSITION_DATE, FINE_AMOUNT, TOTAL_PAID, PENALTY_1, PENALTY_2,
        PENALTY_3, PENALTY_4, PENALTY_5, RP_MULT_OWNER_NO,
        BODY_STYLE, LATITUDE, LONGITUDE, MAR_ID, GIS_LAST_MOD_DTTM
    ) VALUES (
        %s,%s,%s,%s,%s,%s,%s,%s,%s,%s,
        %s,%s,%s,%s,%s,%s,%s,%s,%s,%s,
        %s,%s,%s,%s,%s,%s,%s,%s,%s
    )
    """

    for chunk_df in pd.read_csv(file_name, chunksize=chunk_size):

        chunk_df["ISSUE_DATE"] = pd.to_datetime(chunk_df["ISSUE_DATE"], errors="coerce")
        chunk_df["DISPOSITION_DATE"] = pd.to_datetime(chunk_df["DISPOSITION_DATE"], errors="coerce")
        chunk_df["GIS_LAST_MOD_DTTM"] = pd.to_datetime(chunk_df["GIS_LAST_MOD_DTTM"], errors="coerce")

        #  CLEANING — removes BOTH NaN and "nan" strings
        chunk_df = chunk_df.replace({pd.NA: None, "nan": None, "NaN": None, "": None})
        chunk_df = chunk_df.astype(object).where(pd.notnull(chunk_df), None)

        violations_data = chunk_df[
            ["OBJECTID","LOCATION","XCOORD","YCOORD","ISSUE_DATE","ISSUE_TIME",
             "ISSUING_AGENCY_CODE","ISSUING_AGENCY_NAME","ISSUING_AGENCY_SHORT",
             "VIOLATION_CODE","VIOLATION_PROCESS_DESC","PLATE_STATE",
             "ACCIDENT_INDICATOR","DISPOSITION_CODE","DISPOSITION_TYPE",
             "DISPOSITION_DATE","FINE_AMOUNT","TOTAL_PAID","PENALTY_1","PENALTY_2",
             "PENALTY_3","PENALTY_4","PENALTY_5","RP_MULT_OWNER_NO",
             "BODY_STYLE","LATITUDE","LONGITUDE","MAR_ID","GIS_LAST_MOD_DTTM"]
        ].values.tolist()

        db_cursor.executemany(violations_sql, violations_data)
```

```python
        db_cursor.executemany(violations_sql, violations_data)
        db_conn.commit()

        print(f"Inserted {len(violations_data)} rows from {file_name}")

    db_cursor.close()
    db_conn.close()
    print("Finished:", file_name)

# ---------------- MAIN ----------------
if __name__ == "__main__":

    load_weather()

    violation_files = [
        "Moving_Violations_Issued_in_September_2024.csv",
        "Moving_Violations_Issued_in_October_2024.csv",
        "Moving_Violations_Issued_in_November_2024.csv",
        "Moving_Violations_Issued_in_December_2024.csv",
        "Moving_Violations_Issued_in_January_2025.csv",
        "Moving_Violations_Issued_in_February_2025.csv",
        "Moving_Violations_Issued_in_March_2025.csv"
    ]

    for f in violation_files:
        load_violations(f)

    print("ETL PROCESS COMPLETED SUCCESSFULLY")
```

- RUN the python file in the **VM termina**l to Load the Cleaned & Structured data into the MySQL Tables.

```
nageshasnn1@dc-etl-vm:~/dc_project$ python3 etl.py
Reading Weather Data: Washington, DC, Sept 2024 to Nov 2025.xlsx
Weather Data Imported & Loaded successfully
Loading violations: Moving_Violations_Issued_in_September_2024.csv
Inserted 5000 rows from Moving_Violations_Issued_in_September_2024.csv
Inserted 5000 rows from Moving_Violations_Issued_in_September_2024.csv
Inserted 5000 rows from Moving_Violations_Issued_in_September_2024.csv
Inserted 5000 rows from Moving_Violations_Issued_in_September_2024.csv
Inserted 5000 rows from Moving_Violations_Issued_in_September_2024.csv
```

```
Inserted 5000 rows from Moving_Violations_Issued_in_March_2025.csv
Inserted 5000 rows from Moving_Violations_Issued_in_March_2025.csv
Inserted 5000 rows from Moving_Violations_Issued_in_March_2025.csv
Inserted 5000 rows from Moving_Violations_Issued_in_March_2025.csv
Inserted 5000 rows from Moving_Violations_Issued_in_March_2025.csv
Inserted 5000 rows from Moving_Violations_Issued_in_March_2025.csv
Inserted 5000 rows from Moving_Violations_Issued_in_March_2025.csv
Inserted 5000 rows from Moving_Violations_Issued_in_March_2025.csv
Inserted 5000 rows from Moving_Violations_Issued_in_March_2025.csv
Inserted 1839 rows from Moving_Violations_Issued_in_March_2025.csv
Finished: Moving_Violations_Issued_in_March_2025.csv
ETL PROCESS COMPLETED SUCCESSFULLY
nageshasnn1@dc-etl-vm:~/dc_project$
```

9. Data Update and Maintenance Strategy:

The system supports two safe update approaches:

Incremental Monthly Updates:

- Upload only the latest month's CSV file.
- Run the ETL script.
- Existing records are protected by the primary key.

Only new records are inserted.

Full Data Reload:

- Clear the tables using:

  ```
  TRUNCATE TABLE weather_daily;
  TRUNCATE TABLE moving_violations;
  ```

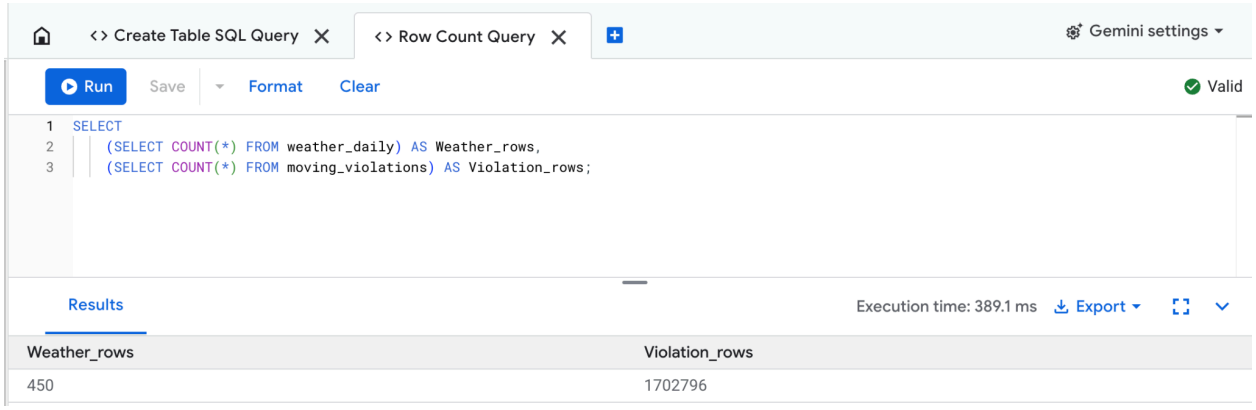
- Run the ETL script again for a full refresh.

Both approaches are useful for data integrity and repeatability.

10. Data Verification and Validation:

After execution of the ETL pipeline, the following verification queries were run:

```
SELECT COUNT(*) FROM weather_daily;
SELECT COUNT(*) FROM moving_violations;
```



Verified Record Counts:

- Weather records: ~450
- Moving violations records: ~1,700,000+

11. SQL Analysis:

After integration, the database successfully supported all required analytical queries:

11.1. Question A:

Query-Question A

Gemini settings

Run Save Format Clear Valid

```sql
# Question A: List every agency that has issued moving violations tickets for each month.
SELECT
    DATE_FORMAT(ISSUE_DATE, '%Y-%m') AS YearMonth,
    ISSUING_AGENCY_NAME,
    COUNT(*) AS Total_Tickets
FROM moving_violations
GROUP BY
    DATE_FORMAT(ISSUE_DATE, '%Y-%m'),
    ISSUING_AGENCY_NAME
ORDER BY
    YearMonth,
    Total_Tickets DESC;
```

Results

Execution time: 9.1 s Export

YearMonth	ISSUING_AGENCY_NAME	Total_Tickets
2024-09	REGIONAL OPERATIONS COMMAND-NORTH	274698
2024-09	METROPOLITAN POLICE DPT-DISTRICT 2	516
2024-09	SPECIAL OPERATION DIV & TRAFFIC DIV	401
2024-09	METROPOLITAN POLICE DPT-DISTRICT 3	395
2024-09	METROPOLITAN POLICE DPT-DISTRICT 1	368
2024-09	METROPOLITAN POLICE DPT-DISTRICT 4	337
2024-09	METROPOLITAN POLICE DPT-DISTRICT 5	312
2024-09	UNITED STATES CAPITOL POLICE	221
2024-09	METROPOLITAN POLICE DPT-DISTRICT 6	200

Rows per page: 20 1 – 20 of 162

11.2. Question B:

Query-Question A Query-Question B

Gemini settings

Run Save Format Clear Valid

```sql
# Question 2: How many tickets have been issued by agencies since October 1, 2024?
SELECT
    ISSUING_AGENCY_NAME,
    COUNT(*) AS Total_Tickets
FROM moving_violations
WHERE ISSUE_DATE >= '2024-10-01'
GROUP BY ISSUING_AGENCY_NAME
ORDER BY Total_Tickets DESC;
```

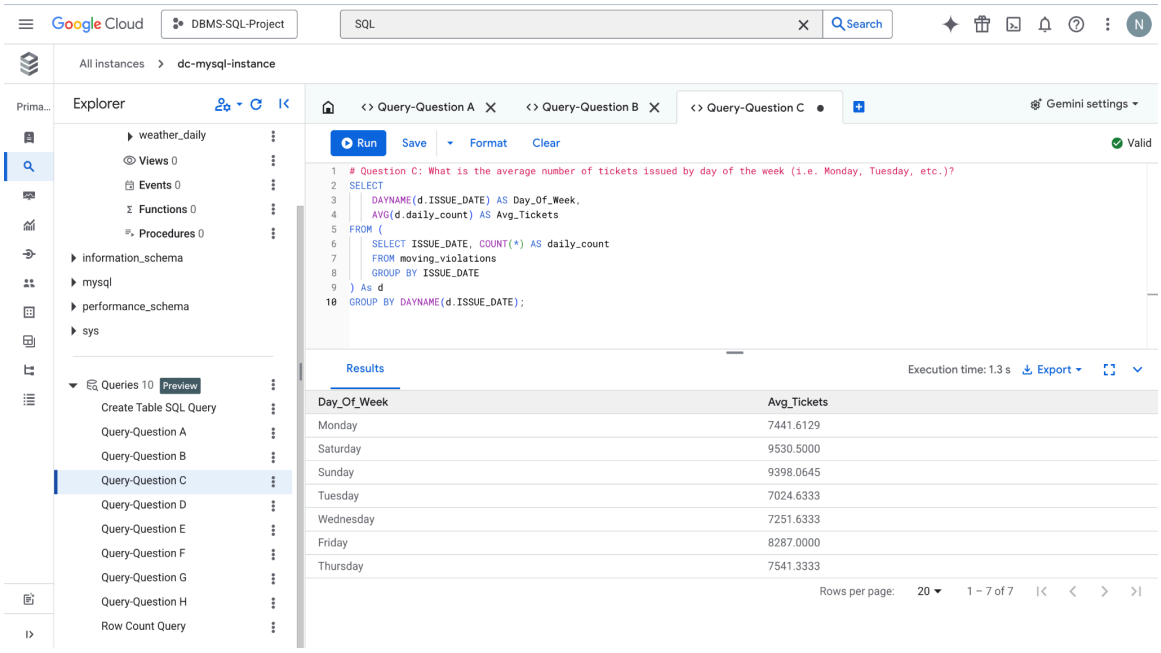
Results

Execution time: 3.2 s Export

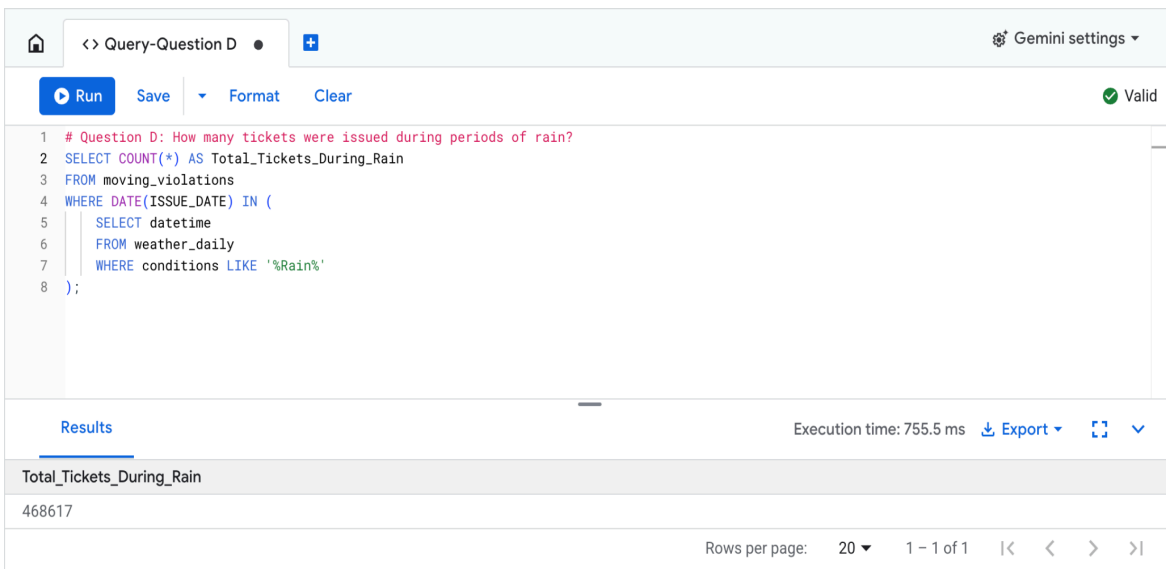
ISSUING_AGENCY_NAME	Total_Tickets
REGIONAL OPERATIONS COMMAND-NORTH	1402752
SPECIAL OPERATION DIV & TRAFFIC DIV	4921
METROPOLITAN POLICE DPT-DISTRICT 2	2571
METROPOLITAN POLICE DPT-DISTRICT 1	2014
METROPOLITAN POLICE DPT-DISTRICT 3	1973
METROPOLITAN POLICE DPT-DISTRICT 7	1807
METROPOLITAN POLICE DPT-DISTRICT 4	1658
METROPOLITAN POLICE DPT-DISTRICT 6	1500
METROPOLITAN POLICE DPT-DISTRICT 5	1258

Rows per page: 20 1 – 20 of 32

11.3. Question C:



11.4. Question D:



11.5. Question E:

Query-Question D | Query-Question E | Gemini settings

Run | Save | Format | Clear | Valid

```sql
# Question E: What was the total precipitation for each month?
SELECT
    DATE_FORMAT(datetime, '%Y-%m') AS YearMonth,
    SUM(precip) AS Total_Precipitation
FROM weather_daily
GROUP BY DATE_FORMAT(datetime, '%Y-%m')
ORDER BY YearMonth;
```

Results — Execution time: 3.0 ms — Export

YearMonth	Total_Precipitation
2024-09	2.100999986520037
2024-10	1.3490000013262033
2024-11	1.546000026166439
2024-12	2.577999962377362
2025-01	1.9029999617487192
2025-02	2.623999967239797
2025-03	2.1260000149486586
2025-04	3.341000035405159
2025-05	7.267999995383434

Rows per page: 20 — 1 – 15 of 15

11.6. Question F:

Query-Question D | Query-Question E | Query-Question F | Gemini settings

Run | Save | Format | Clear | Valid

```sql
# Question F: What is the total fine issued each month for vehicles traveling more than 10 mph over the speed limit?
SELECT
    DATE_FORMAT(ISSUE_DATE, '%Y-%m') AS YearMonth,
    SUM(FINE_AMOUNT) AS Total_Fines
FROM moving_violations
WHERE VIOLATION_PROCESS_DESC LIKE '%MPH%'
  AND FINE_AMOUNT IS NOT NULL
GROUP BY DATE_FORMAT(ISSUE_DATE, '%Y-%m')
ORDER BY YearMonth;
```

Results — Execution time: 4.2 s — Export

YearMonth	Total_Fines
2024-09	25345050.00
2024-10	23537050.00
2024-11	23877800.00
2024-12	23242000.00
2025-01	19057800.00
2025-02	9952700.00
2025-03	23464600.00

Rows per page: 20 — 1 – 7 of 7

11.7. Question G:

Query-Question D | Query-Question E | Query-Question F | Query-Question G | Gemini settings

Run | Save | Format | Clear | Valid

```
# Question G: What is the average number of tickets written for each hour of a standard day (i.e. from 7:00-8:00am, 8:00-9:00am, etc.)?
SELECT
    issue_hour AS hour_of_day,
    AVG(hourly_count) AS avg_tickets
FROM (
    SELECT
        DATE(ISSUE_DATE) AS issue_date_only,
        HOUR(ISSUE_DATE) AS issue_hour,
        COUNT(*) AS hourly_count
    FROM moving_violations
    GROUP BY DATE(ISSUE_DATE), HOUR(ISSUE_DATE)
) AS dayhours
GROUP BY issue_hour
ORDER BY hour_of_day;
```

Results — Execution time: 1.4 s — Export

hour_of_day	avg_tickets
4	9164.6744
5	7317.0720

Rows per page: 20 — 1 – 2 of 2

11.8. Question H:

Query-Question G | Query-Question H | Gemini settings

Run | Save | Format | Clear | Valid

```
# Question H: Compare tickets associated with accidents on rainy days vs non-rainy days.
SELECT
    CASE
        WHEN wd.conditions LIKE '%Rain%' THEN 'Rainy Days'
        ELSE 'Non-Rainy Days'
    END AS Weather_type,
    COUNT(*) AS Total_Accident_Tickets
FROM moving_violations mv
JOIN weather_daily wd
    ON DATE(mv.ISSUE_DATE) = wd.datetime
WHERE mv.ACCIDENT_INDICATOR = 'Y'
GROUP BY Weather_type;
```

Results — Execution time: 689.2 ms — Export

Weather_type	Total_Accident_Tickets
Non-Rainy Days	1555
Rainy Days	666

Rows per page: 20 — 1 – 2 of 2

12. Final Outcome:

This project successfully delivered:

- A cloud-based relational data warehouse

- A fully automated ETL pipeline
- Safe handling of millions of records
- A scalable update strategy
- A verified dataset for SQL-driven analysis